

Referral Copilot

Defesa da solução — problema, decisões de engenharia, trade-offs e evidências

1. O problema e a tese

Vagas técnicas se preenchem por rede, mas a rede da empresa está fragmentada nas contas pessoais de cada pessoa do time — e ninguém vai entregar sua lista de contatos ao RH. O produto resolve essa tensão invertendo a posse do dado: **cada pessoa mantém a própria rede privada**, a plataforma cruza essa rede com as vagas abertas e só o que o dono confirma como indicação se torna visível ao administrador.

A tese técnica é que a maior parte do valor vem de **ranqueamento explicável sobre dados que a pessoa já tem**, não de um modelo caro. A IA entra onde agrega — interpretar descrição de vaga, reranquear os melhores resultados, resumir a composição da rede — sempre com caminho determinístico de fallback.

120

TESTES AUTOMATIZADOS

374

CONEXÕES REAIS MAPEADAS

4

ESPECIALISTAS NA FILA

0

CREDENCIAIS
ARMAZENADAS

2. Decisões de engenharia e trade-offs

DECISÃO 1 · COLETA

Conector local (extensão) em vez de navegador remoto

Contexto: a primeira implementação usava navegador remoto (Browserless, depois Anchor Browser): o usuário logava numa sessão remota e um worker coletava. Funcionava no papel e a arquitetura era elegante — zero instalação.

O que o piloto real mostrou: fricção alta e inaceitável. O tier gratuito do Browserless não expõe a URL interativa; o login numa janela remota gera desconfiança legítima (o usuário digita a senha numa tela que não é a dele); e verificação em duas etapas trava o fluxo.

Decisão: abandonar o remoto e adotar a extensão local, onde o login acontece na sessão do próprio usuário.

Trade-off aceito: exige instalação uma única vez — perdemos o "zero setup" e ganhamos confiança, robustez e um modelo de privacidade defensável (nenhuma credencial trafega).

Consequência assumida: depois de validado, o fluxo remoto foi *removido* do código (5.279 linhas, 63 arquivos), em vez de mantido como caminho morto atrás de flag.

DECISÃO 2 · PRIVACIDADE

O administrador não tem rota de leitura da rede dos membros

Não é regra de UI: **não existe endpoint** que liste os contatos privados de um membro para o administrador. O consentimento é registrado por indicação, com os campos que foram autorizados. Isso torna a promessa de privacidade verificável no código, não apenas na tela.

DECISÃO 3 · IA

Heurística determinística primeiro; LLM só onde ela não alcança

Capital de rede e classificação de área são **regras versionadas e explicáveis**, sem LLM: custo zero, resposta imediata, resultado reproduzível e testável. A LLM entra em três pontos — interpretar a vaga, reranquear os melhores candidatos e resumir a rede — e **todos têm fallback determinístico**. Se o provedor cair, o produto continua funcionando com qualidade menor, nunca quebrado.

Trade-off: heurística erra em casos raros e exige manutenção de regras. Mitigação: override manual de área pelo administrador, que sempre prevalece sobre a inferência.

DECISÃO 4 · DADOS ENVIADOS À LLM

Minimização por desenho

O reranking usa identificadores opacos, sem nome, telefone, e-mail ou URL. Os insights de rede enviam **apenas agregados** (distribuição por área, cobertura de contato, faixa de sinal) — nenhum nome sai da plataforma. A análise de vaga não contém dado pessoal.

DECISÃO 5 · ASSINCRONIA

Fila no próprio PostgreSQL, não uma dependência nova

O trabalho pesado sai do request e vai para um worker separado, com fila em Postgres usando `SKIP LOCKED`, dependências entre tarefas (DAG), chave de idempotência, orçamento de tokens, timeout, *lease* de 90s renovado por heartbeat e três tentativas com backoff.

Por que não Redis/SQS: o volume não justifica outro serviço para operar e pagar; o Postgres já é dependência e dá transação junto com o dado. **Trade-off:** não escala para milhões de tarefas — é a escolha certa para o estágio, e a troca depois é local (a interface da fila é isolada).

O lease curto existe por um motivo concreto: quando um deploy mata o worker no meio de uma tarefa, ela volta para a fila em ~2 minutos em vez de esperar o timeout inteiro.

DECISÃO 6 · PERSISTÊNCIA

Sincronização incremental, nunca destrutiva

A gravação é *upsert* por URL do perfil. Uma coleta parcial (aba fechada no meio, rede instável) atualiza o que veio e **nunca reduz** a rede já mapeada. Antes dessa mudança, uma execução ruim podia encolher o total — o tipo de bug que destrói confiança no produto.

3. Arquitetura em uma página

CAMADA	ESCOLHA	RAZÃO
Web	Next.js 16 + React 19, Route Handlers	Um só runtime para UI e API reduz superfície de operação; renderização no servidor mantém a autorização perto do dado.
Dados	PostgreSQL com <code>pg</code> , FKs, índices, transações	Integridade referencial e transação onde o domínio exige (aceitar convite, gravar consentimento).
Migração	Script idempotente no <code>start</code>	Deploy sem etapa manual; seguro para rodar em toda subida.
Assíncrono	Worker separado consumindo fila Postgres	Isola custo e latência de IA do caminho do usuário; escala independente.
Validação	Zod em toda fronteira externa	Dado de extensão, upload e payload de tarefa nunca entram sem contrato.
Coleta	Extensão Manifest V3 com funções puras testadas	A lógica de extração é testável fora do navegador, sem DOM.
Contato	WhatsApp click-to-chat oficial	Sem automação de DOM e sem disparo em massa: o humano revisa e envia.

Segurança aplicada

- Cookies de sessão assinados, `httpOnly`, `sameSite=lax` e `secure` em produção.
- Chave administrativa comparada em tempo constante (evita ataque por timing).
- Convites: token aleatório com **apenas o hash persistido**; consumo dentro de transação com bloqueio de linha, o que impede aceite duplicado em concorrência.
- Contatos isolados por dono, sem rota administrativa de listagem.

4. Como sei que funciona (evidência, não opinião)

VERIFICAÇÃO	RESULTADO
Suíte automatizada (Vitest)	120 testes passando em 28 arquivos
Tipagem estrita (<code>tsc --noEmit</code>)	Sem erros
Build de produção (<code>next build</code>)	Passa; mapa de rotas conferido
Piloto real de coleta	374 conexões reais mapeadas de ponta a ponta
Serviço web em produção	<code>HTTP 200</code>
Worker em produção	Log <code>event="worker_started"</code> após migração

A extração do LinkedIn foi validada **contra o HTML real** da página de conexões antes de virar código de produção, e as funções de extração (nome e cargo) são puras e cobertas por teste — inclusive casos difíceis como nomes com sufixo ("Nome, LLB, MBA") e cartões sem cargo.

Dois bugs que valem contar

Lista virtualizada: a coleta parava em ~10 contatos. Causa: o LinkedIn mantém só ~10 cartões no DOM e recicla ao rolar. Solução: rolar em passos pequenos coletando a cada passo, normalizando a URL para `/in/<slug>` — validado no console antes de portar.

Nome colado no cargo: a extração usava `textContent`, que concatena sem quebra de linha. Passou a usar as linhas do `innerText` do cartão. O sintoma aparecia na tela; a causa estava na semântica de duas APIs parecidas do DOM.

5. Limites conhecidos (o que eu *não* vou defender)

Termos de uso do LinkedIn. A plataforma não usa API oficial do LinkedIn — lê o que o usuário já vê, na sessão dele, por ação explícita dele. Isso reduz o risco (sem credencial em servidor, sem acesso a dado não visível, sem operação em massa e com pausas que respeitam o ritmo da página), mas **não elimina** a fricção com os termos de uso, que restringem coleta automatizada. Para uso comercial em escala, o caminho correto é parceria/API oficial ou importação manual do arquivo de conexões — o produto já aceita esse arquivo.

- **Profundidade do perfil:** hoje se captura nome, cargo e URL. Formação e histórico só entram se preenchidos manualmente ou pelo arquivo importado. Aprofundar exigiria visitar cada perfil — mais tempo e mais exposição, então ficou fora de escopo.
- **Classificação heurística:** cobre bem tech, dados, produto e finanças; áreas de nicho podem cair no rótulo genérico. Por isso existe o override manual.
- **Fila em Postgres:** adequada para o volume atual, não para milhões de tarefas.
- **Google Contacts:** declarado como implementação futura e desativado — preferi um "não disponível" honesto a uma integração meia-boca.
- **Duas tabelas órfãs** do fluxo remoto continuam no banco, vazias e sem código que as leia. A remoção foi deixada para uma janela própria, porque *drop* é irreversível.

6. Perguntas difíceis, respondidas

"Por que não usar a API oficial do LinkedIn?"

A API oficial não expõe a lista de conexões de um membro para uma aplicação terceira — esse dado simplesmente não está disponível por lá. Como o produto depende exatamente dele, as opções reais eram: importação manual do arquivo de conexões (aceita pelo produto) ou leitura assistida na sessão do usuário. Escolhi oferecer as duas, deixando o limite explícito.

"Isso é só um wrapper de LLM?"

Não. O ranqueamento central é determinístico e explicável; a LLM reranqueia e resume. Desligando todos os provedores, o produto continua entregando recomendações com evidência — só perde nuance. A evidência disso é que **a suíte inteira roda sem nenhum provedor configurado**: parsing de vaga, fit de candidato, score de indicação, capital de rede e classificação de área são todos testados sem chamar LLM. (Um teste que simule explicitamente a queda do provedor em produção é uma lacuna reconhecida da cobertura.)

"Como você evita viés na recomendação?"

Três travas deliberadas: (1) os sinais de capital de rede têm contribuição limitada e servem *apenas* à hipótese de que alguém conheça bons candidatos, nunca ao fit do candidato; (2) a ausência de formação ou empresa reconhecida **nunca** reduz pontuação; (3) toda recomendação carrega a evidência textual que a sustenta, então a decisão humana é auditável. A plataforma não decide contratação.

"E se o membro sair da empresa?"

A rede nunca foi da empresa — é do membro. Sai com ele. O que a empresa retém são as indicações que foram explicitamente consentidas, com o registro do que foi autorizado. É o desenho correto do ponto de vista ético e o que torna o produto adotável.

"Por que apagar o fluxo remoto em vez de deixar atrás de uma flag?"

Porque código morto atrás de flag tem custo real: aumenta a superfície de segurança (rotas e segredos ativos), confunde quem lê depois, e a documentação envelhece apontando para um caminho que não existe. Depois de decidir a direção, mantive apenas o que estava vivo — e verifiquei o corte com build, testes e logs de produção antes de considerar concluído.

"Qual seria seu próximo passo com mais tempo?"

Nesta ordem: (1) medir conversão real do funil indicação→contratação, que é a métrica que valida a tese; (2) enriquecimento opcional de perfil com consentimento por contato; (3) publicar a extensão na loja para eliminar a instalação em modo desenvolvedor; (4) API oficial do WhatsApp quando o volume justificar.